UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

Facultad de Ciencias

Sistemas Operativos

Planificador de Tareas (Scheduler) Round Robin y Prioridades

Práctica 4

Presenta:
Rojo Peña Manuel Ianluck

Profesor:
Javier León Cotonieto

Ayudante laboratorio:
Adrián Martínez Manzo

Grupo: 7004, 2026-2

Fecha de entrega:
17 de marzo, 2026

Introducción

Planteamiento del Problema

En la práctica anterior el cambio de contexto entre tareas dependía de un evento externo (un botoncito), lo que nos imposibilita la ejecución concurrente autónoma. Y como sabemos, un **sistema operativo** real necesita garantizar que ninguna tarea monopolice el CPU de forma indefinida, distribuyendo el tiempo de cómputo de manera justa y predecible.

En esta práctica resolvemos ese problema en dos fases:

1. **Fase A**: Reemplazamos el botoncito por un temporizador de hardware (*SysTick*) que genera interrupciones cada 10 ms y habilitamos la comunicación por USB para "encolar" tareas desde la terminal.

2. **Fase B**: Agregar un campo de *peso* (lo llamaremos `quantum`) al **TCB** para implementar ***Round Robin Ponderado*** (WRR) y asignar prioridades en tiempo real.

Objetivos

- Implementar la expropiación automática de CPU mediante el temporizador *SysTick* sobre la Raspberry Pi Pico.

- Diseñar el **Bloque de Control de Tareas** (**TCB**) y gestionar los estados `DORMANT`, `READY` y `RUNNING`.

- Implementar el algoritmo Round Robin estándar y evolucionarlo hacia un *Round Robin Ponderado* que permita asignar cuantas de tiempo variables.

- Demostrar el impacto de los retardos por *busy waiting* en la planificación concurrente y comprender su efecto en la percepción visual.

Conceptos

Bloque de Control de Tareas (TCB)

El **Task Control Block** (**TCB**) es la estructura de datos central de cualquier planificador, esta concentra toda la información que el Kernel necesita para suspender y reanudar un proceso como si nunca hubiera sido interrumpido: el apuntador a la pila (`sp`), el estado de ciclo de vida (`state`), el espacio de memoria exclusivo de la tarea (`stack`) y la dirección de la función que la implementa (`entry_point`).

Round Robin (RR)

Round Robin es el algoritmo de planificación más simple con equidad garantizada, mantiene una cola circular de procesos `READY`; cuando el temporizador expira, la tarea actual es expulsada y la siguiente de la cola recibe el CPU durante exactamente un *quantum* de tiempo. La ecuación del reparto es directa: si hay N tareas con el mismo *quantum*, cada una recibe $\frac{1}{N}$ del tiempo total de CPU.

Round Robin Ponderado (WRR)

El *Round Robin Ponderado* extiende la equidad del ***RR*** permitiendo asignar a cada proceso un *quantum* q_i que actúa como multiplicador de quantum. El reparto de CPU se convierte en:

$$\mathrm{CPU}_i = \frac{q_i}{\sum_{j=1}^{N} q_j}$$

Una tarea con $q = 3$ ejecuta durante 3 *ticks* del *SysTick* (30 ms) mientras que una con $q = 1$ solo ejecuta 10 ms por turno.

SysTick y PendSV

El *SysTick* es un temporizador descendente, al llegar a cero genera una excepción de prioridad alta (`isr_systick`). Para nuestra implementación de `ISR` no realizamos el cambio de contexto directamente, sino que activamos la excepción **PendSV** (como en la anterior práctica), que tiene la prioridad más baja del sistema y se ejecuta únicamente cuando no hay otras excepciones pendientes, garantizando que el cambio de contexto ocurra en el momento más seguro posible.

Syscall `sys_exit()` y PendSV

Cuando una tarea finita termina su trabajo invoca `sys_exit()`, esta función emite una instrucción `SVC` que eleva el privilegio al modo Kernel, donde `k_task_exit()` marca

el estado de la tarea como `DORMANT` y activa inmediatamente **PendSV** para que el planificador elija la siguiente tarea sin esperar el siguiente tick.

Fase A: Round Robin y Gestión de Tareas

Estructura TCB (`scheduler.h`)

Esta estructura base define los cuatro campos mínimos para gestionar una tarea:

```c
// Estados posibles de una tarea
typedef enum {
  DORMANT, // Inactiva, no esta en la cola
  READY,   // Lista para ejecutarse
  RUNNING  // Ejecutandose actualmente
} task_state_t;

// Task Control Block
typedef struct {
  uint32_t *sp;                  // Puntero de Pila actual
  task_state_t state;            // Estado de la tarea
  uint32_t stack[STACK_SIZE];    // Espacio de memoria de la pila
  void (*entry_point)(void);     // Funcion de la tarea
  uint8_t quantum; //priority quantum
  uint8_t remaining;  //quantum countdown
} tcb_t;
```

Código 1: TCB base en `scheduler.h`

El campo `sp` permite congelar y restaurar el contexto exacto de la tarea, `stack[]` es la memoria privada de cada proceso y evita que los marcos de pila de distintas tareas se sobrepongan.

Inicialización del Stack (`init_task_stack()`)

Para que el manejador **PendSV** pueda restaurar el contexto de una tarea recién creada, su pila debe pre-cargarse con un *frame* ficticio que imita el que el hardware genera al entrar en una excepción:

```c
/**
 * @brief Initialize the task stack so it looks like an interrupted
   context.
 * @param id Task index in the task table.
 */
void init_task_stack(int id) {
  uint32_t *stack_top = tasks[id].stack + STACK_SIZE;

  // Hardware stack frame (simulated)
  *(--stack_top) = 0x01000000; // xPSR (Thumb bit enabled)
```

```c
  *(--stack_top) = (uint32_t)tasks[id].entry_point; // PC (Program Counter)
  *(--stack_top) = 0xFFFFFFFD; // LR (Return to Thread mode using PSP)
  *(--stack_top) = 0; // R12
  *(--stack_top) = 0; // R3
  *(--stack_top) = 0; // R2
  *(--stack_top) = 0; // R1
  *(--stack_top) = 0; // R0

  // Software stack frame (simulated, R4-R11)
  for (int i = 0; i < 8; i++) {
    *(--stack_top) = 0;
  }

  tasks[id].sp = stack_top;
  tasks[id].state = READY;
  //Load initial quantum
  tasks[id].remaining = tasks[id].quantum;
}
```

Código 2: Frame ficticio de hardware en `init_task_stack()`

Al terminar, `tasks[id].sp` apunta a la base del frame y la tarea queda en estado `READY`.

Control por USB (`isr_systick()`)

La función `getchar_timeout_us(0)` devuelve el carácter disponible de inmediato o `PICO_ERROR_TIMEOUT` si no hay ninguno, garantizando que la `ISR` nunca se bloquee:

```c
int c = getchar_timeout_us(0);
if (c >= '1' && c <= '4') {
    int idx = c - '1';
    if (tasks[idx].state == DORMANT) {
        init_task_stack(idx);
        printf("Tarea %d agregada a la cola.\n", idx + 1);
    }
}
// Disparar el cambio de contexto
*(volatile uint32_t *)0xE000ED04 = (1 << 28);
```

Código 3: Polling no bloqueante de la UART USB en `isr_systick()`

Tareas de Usuario `finite` e `infinite` (`user_app.c`)

Implementamos dos nuevas tareas que ejercitan los dos posibles ciclos de vida:

```c
/**
 * @brief Infinite task that rapidly blinks LED_1.
 */
```

```c
void task3_infinite(void)
{
  sys_gpio_dir(LED_1, 1);
  while (1)
    {
      sys_gpio_set(LED_1, 1);
      delay_cycles_exact(41700000 / 40);
      sys_gpio_set(LED_1, 0);
      delay_cycles_exact(41700000 / 40);
    }
}
```

Código 4: Tarea infinita: parpadeo rápido de LED en `task3_infinite()`

```c
/**
 * @brief Finite task that blinks an LED 5 times and then terminates.
 */
void task4_finite(void)
{
  sys_gpio_dir(LED_3, 1);

  for (int i = 0; i < 15; i++)
    {
      sys_gpio_set(LED_3, 1);
      delay_cycles_exact(41700000 / 10);
      sys_gpio_set(LED_3, 0);
      delay_cycles_exact(41700000 / 10);
    }
  //Life cycle completed, then we notify the Kernel this task finished
  sys_exit();
}
```

Código 5: Tarea finita: 15 parpadeos y `sys_exit()` en `task4_finite()`

Syscall `sys_exit()` (`k_task_exit()`)

Cuando el Kernel intercepta la syscall `EXIT`, marca la tarea como `DORMANT` y fuerza un cambio de contexto inmediato sin esperar al siguiente tick del *SysTick*:

```c
/**
 * @brief Handle the EXIT syscall by terminating the current task.
 */
void k_task_exit(void) {
  if (current_task != -1) {
    tasks[current_task].state = DORMANT;
    printf("Tarea %d terminada.\n", current_task + 1);
    // Force an immediate context switch
    *(volatile uint32_t *)0xE000ED04 = (1 << 28);
  }
}
```

Código 6: Gestión del fin de tarea en `k_task_exit()`

Algoritmo Round Robin (`schedule()`)

La función `schedule()` recorre el arreglo circular de tareas buscando la siguiente en estado `READY`, garantizando que ninguna sea ignorada indefinidamente:

```
int next = current_task;
int tries = 0;
do {
    next = (next + 1) % MAX_TASKS;
    tries++;
    if (tasks[next].state == READY) break;
} while (tries < MAX_TASKS);

if (tasks[next].state == READY) {
    current_task = next;
    tasks[current_task].state = RUNNING;
    return (uint32_t)tasks[current_task].sp;
}
return current_sp; // Cola vacia: idle
```

Código 7: Búsqueda Round Robin en `schedule()`

Análisis de Concurrencia: Busy Waiting y Time Slicing

Fenómeno de ralentización (*stuttering*)

Al encolar las cuatro tareas con quantum $q = 1$, el *SysTick* divide el CPU en ranuras de 10 ms que se reparten equitativamente, cada tarea recibe únicamente $\frac{1}{N}$ del tiempo total. Con $N = 4$ tareas activas, una animación diseñada para actualizar sus LEDs cada 50 ms real en realidad tarda:

$$t_{\text{efectivo}} = t_{\text{nominal}} \times N = 50\,\text{ms} \times 4 = 200\,\text{ms}$$

El resultado perceptible es que la animación que originalmente debería moverse a 20 cuadros por segundo se reduce a solo 5, produciendo un efecto de parpadeo discontinuo (*stuttering*).

El papel del Busy Waiting

Las tareas utilizan `delay_cycles_exact()`, un bucle que quema ciclos de CPU sin realizar trabajo útil. Desde la perspectiva del planificador la tarea continúa en estado `RUNNING` durante todo ese retardo, impidiendo que otras tareas avancen. Este es el

costo fundamental del *busy waiting* pues, el CPU no puede distinguir entre "una tarea que procesa datos" y "una tarea que espera"; ambas consumen el mismo quantum.

En un **sistema operativo** real este problema se resuelve con llamadas de sistema bloqueantes como `sleep()` que colocan la tarea en estado `BLOCKED` y ceden el CPU voluntariamente. En esta práctica se conserva el busy waiting para ilustrar precisamente este efecto.

Impacto matemático del Time Slicing

La tabla siguiente resume el tiempo efectivo de CPU por turno según el número de tareas activas con *Round Robin* puro ($q = 1$)[1]:

Table 1: Reparto de CPU con Round Robin puro y SysTick = 10 ms

Tareas activas (N)	**Tiempo por turno**	**Factor de lentitud**
1	10 ms	1×
2	10 ms	2×
3	10 ms	3×
4	10 ms	4×

[1] El tiempo por turno no cambia (siempre 10 ms), pero la frecuencia con que cada tarea recibe CPU sí disminuye linealmente con N.

Fase B: Round Robin Ponderado (WRR)

Modificación del TCB

Se añaden dos campos al TCB, el quantum asignado a la tarea y la cuenta regresiva de ticks restantes en el quantum actual.

```c
typedef struct {
    uint32_t    *sp;
    task_state_t state;
    uint32_t     stack[STACK_SIZE];
    void        (*entry_point)(void);
    uint8_t quantum; //priority quantum
    uint8_t remaining;  //quantum countdown
} tcb_t;
```

Código 8: Nuevos campos en `tcb_t` — `scheduler.h`

Lógica WRR en `isr_systick()`

La `ISR` del *SysTick* ya no dispara **PendSV** incondicionalmente, sino que primero decrementa el contador, solo cuando llega a cero expulsa a la tarea:

```c
//Decrement quantum
  if (current_task != -1 && tasks[current_task].state == RUNNING) {
    if (tasks[current_task].remaining > 1) {
      tasks[current_task].remaining--;
      return;   //task keep CPU, NOT launch pendsv
    }
    //Quantum spent: reload next time
    tasks[current_task].remaining = tasks[current_task].quantum;
  }

  // Write to the Interrupt Control and State Register (ICSR)
  *(volatile uint32_t *)0xE000ED04 = (1 << 28);
```

Código 9: Control del quantum en `isr_systick()` — `scheduler.c`

Recarga del quantum en `schedule()`

Cuando el planificador elige una nueva tarea, recarga su contador con su quantum para iniciar el quantum desde el valor correcto:

```c
// If a runnable task was found
if (tasks[next_task].state == READY) {
    current_task = next_task;
    tasks[current_task].state = RUNNING;
    //Reaload quantum with the income one
    tasks[current_task].remaining = tasks[current_task].quantum;
```

```
    return (uint32_t)tasks[current_task].sp;
}
```

Código 10: Recarga del quantum al seleccionar tarea en `schedule()`

Asignación de pesos por terminal USB

También implementamos un mini-parser de dos pasos en la `ISR` que, al recibir la tecla `q`, el Kernel entra en modo de configuración, solicita el identificador de tarea y luego el nuevo peso sin bloquear la `ISR` entre pasos:

```
//USB comms parser state
//0 = idle
//1 = 'q' received, wait id
//2 = id received, wait quantum (weight)
static int cmd_state = 0;
static int cmd_task_id = -1;

if (c == 'q') {
    printf("[Kernel] Peso > selecciona tarea (1-4): ");
    cmd_state = 1;
}
else if (cmd_state == 1 && c >= '1' && c <= '4') {
    cmd_task_id = c - '1';
    cmd_state   = 2;
}
else if (cmd_state == 2 && c >= '1' && c <= '9') {
    tasks[cmd_task_id].weight         = c - '0';
    tasks[cmd_task_id].ticks_remaining = c - '0';
    printf("[Kernel] Tarea %d -> peso=%d (%d ms/quantum).\n",
           cmd_task_id + 1, c - '0', (c - '0') * 10);
    cmd_state = 0;
}
```

Código 11: Parser de comandos de peso en `isr_systick()`

Impacto del peso en el reparto de CPU

En la siguiente tabla ilustramos cómo el **WRR** redistribuye el tiempo cuando `task_2` recibe un quantum elevado mientras las demás mantienen $q = 1$:

Table 2: Reparto de CPU con WRR: Tarea 2 priorizada ($N = 4$ tareas activas)

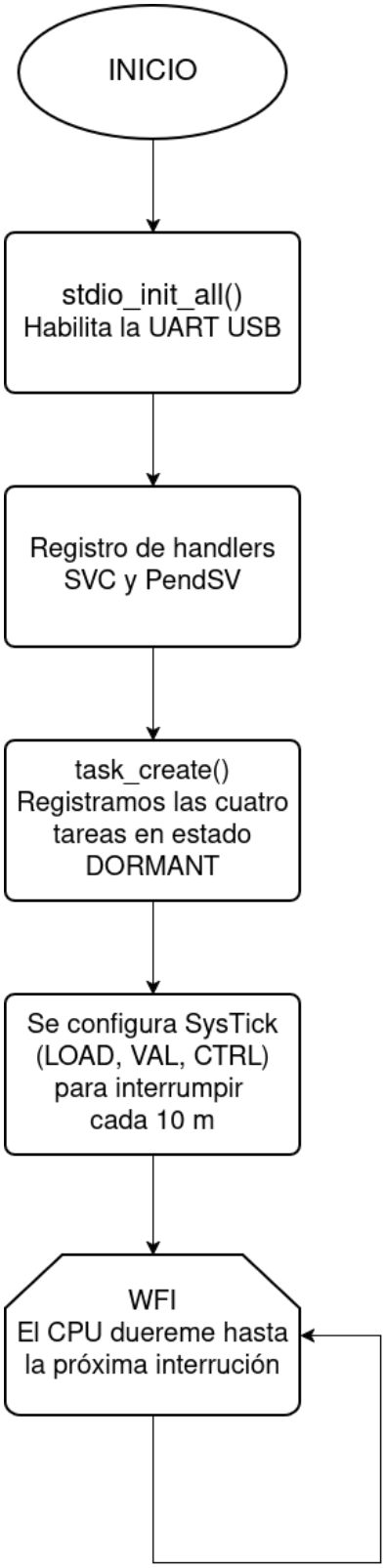
Tarea	**Peso** (w)	**Quantum**	**% de CPU**
1	1	10 ms	$\approx 12.5\%$
2	5	50 ms	$\approx 62.5\%$
3	1	10 ms	$\approx 12.5\%$
4	1	10 ms	$\approx 12.5\%$

El resultado físico es que la animación de la tarea 2 se observa notablemente más fluida mientras las demás tareas se vuelven proporcionalmente más lentas, demostrando el control de prioridades en tiempo real.

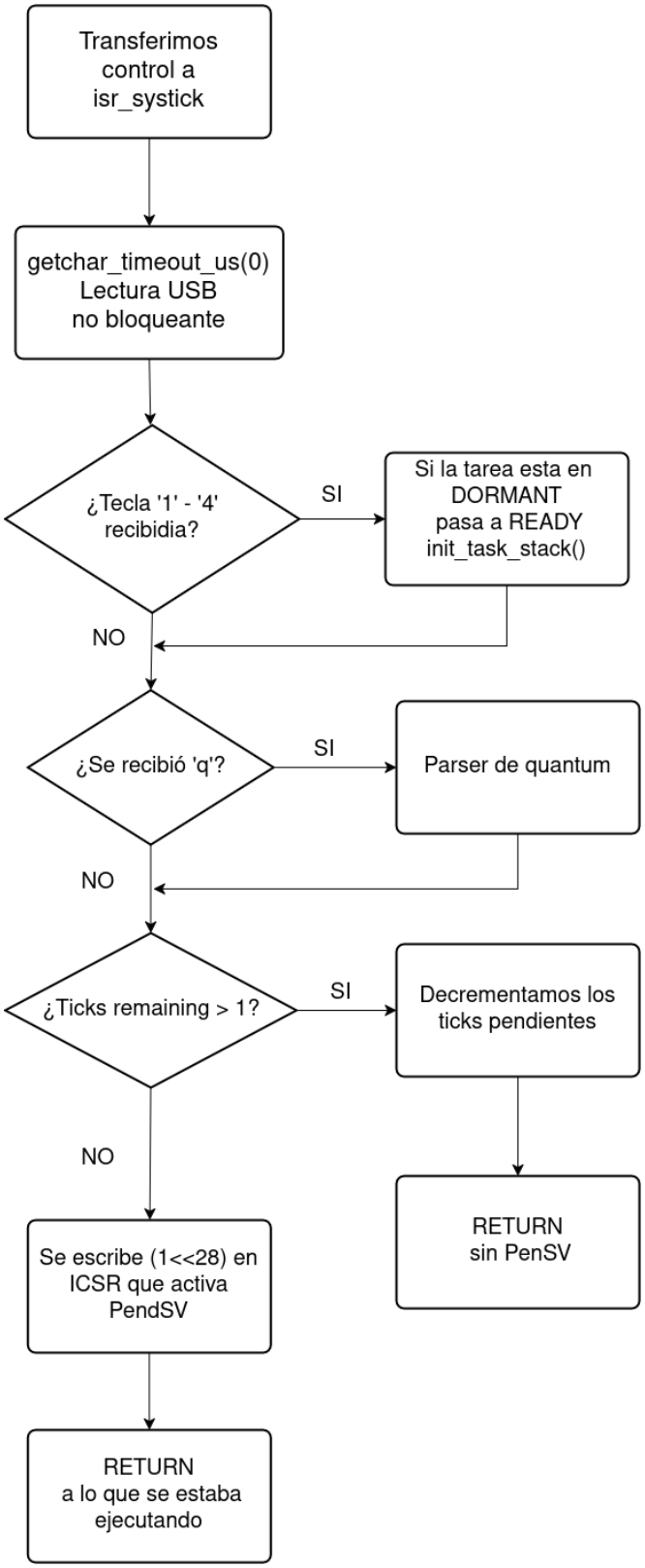
Diagrama de Flujo

El diagrama de flujo del sistema se divide en los tres hilos de ejecución independientes.

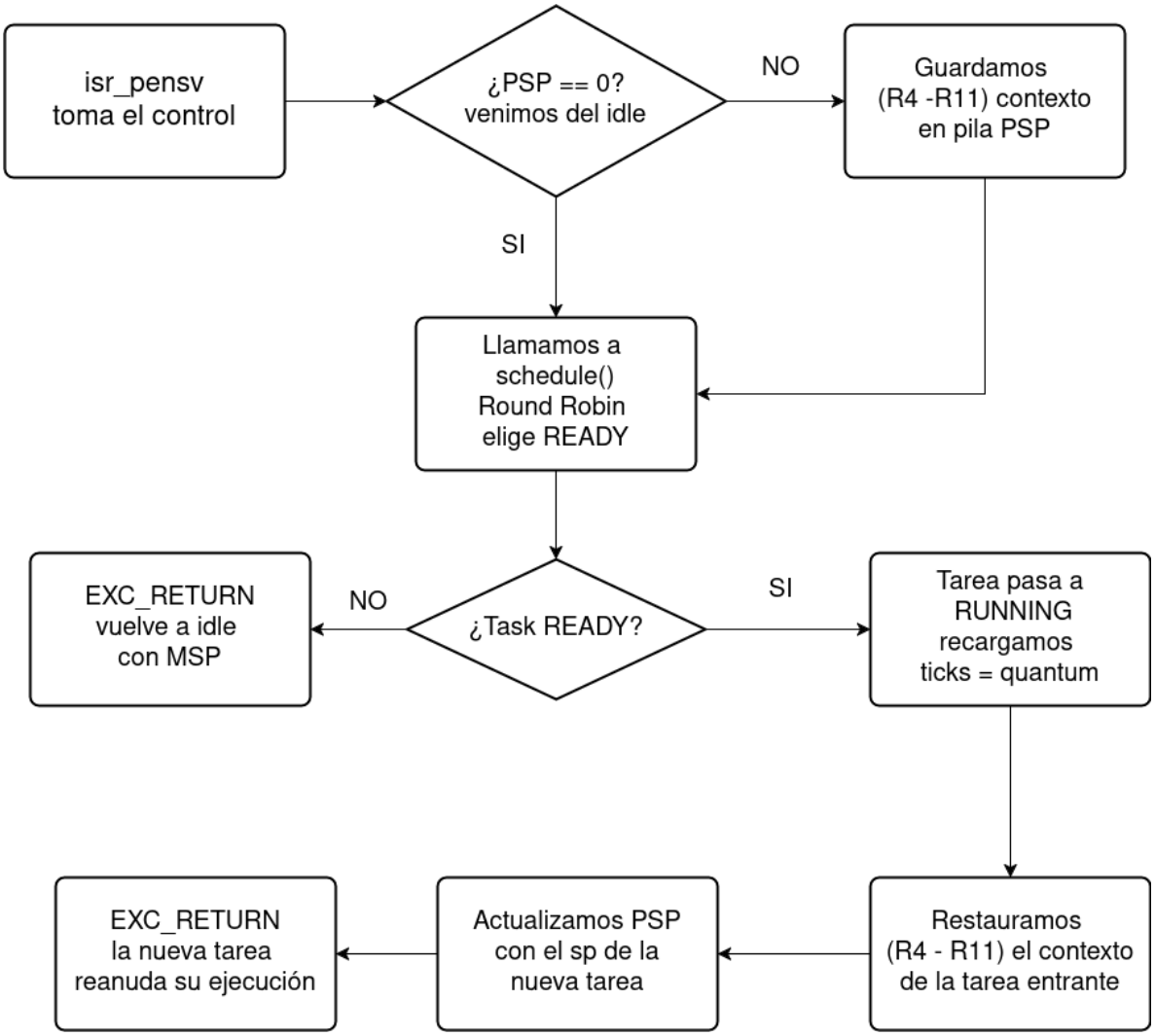
Hilo 1 - Arranque y Bucle Idle (`main.c`)



Hilo 2 - ISR del SysTick (cada 10 ms)



Hilo 3 - Cambio de Contexto (`pendsv.s` + `schedule()`)



Conclusiones

1. **El SysTick es la base del multitarea preemptivo.**

   La sustitución del botón por el temporizador de hardware demostró que la preempción automática es lo que convierte un secuenciador manual en un Sistema Operativo real. Con interrupciones cada 10 ms el Kernel recupera el control del CPU de forma periódica e independiente de la voluntad de las tareas, principio fundamental de todo planificador preemptivo.

2. **El TCB es la abstracción esencial del proceso.**

   Encapsular el puntero de pila, el estado y el punto de entrada en una estructura unificada permitió tratar cada tarea como una entidad independiente y autosuficiente. La pre-carga del frame ficticio en `init_task_stack()` revela que el "contexto inicial" de un proceso no difiere en estructura del contexto de uno interrumpido: el hardware no distingue entre ambos.

3. **El busy waiting es costoso en un entorno concurrente.**

   Los retardos implementados con `delay_cycles_exact()` consumen el quantum completo realizando trabajo nulo, ralentizando la ejecución de todas las tareas en un factor proporcional a N. Esta experiencia justifica por qué los sistemas operativos modernos implementan llamadas bloqueantes (`sleep`, `wait`) que liberan el CPU durante la espera.

4. **El Round Robin Ponderado permite priorización sin inanición.**

   Al asignar un peso w_i mayor a la Tarea 2 se observó físicamente cómo su animación se volvió más fluida mientras las demás tareas recibían menos ciclos. El WRR demostró ser una solución pragmática: garantiza que todas las tareas eventualmente ejecutan (sin inanición) mientras permite influir en la distribución del tiempo.

5. **La interacción serial en tiempo real valida el diseño.**

   El parser de comandos integrado directamente en la ISR, usando variables de estado estáticas en lugar de estructuras bloqueantes, demostró que es posible modificar los parámetros del planificador en caliente (*hot-reconfiguration*) sin detener el sistema, técnica análoga a la que usan los administradores de sistemas para ajustar prioridades con `nice`/`renice` en Linux.